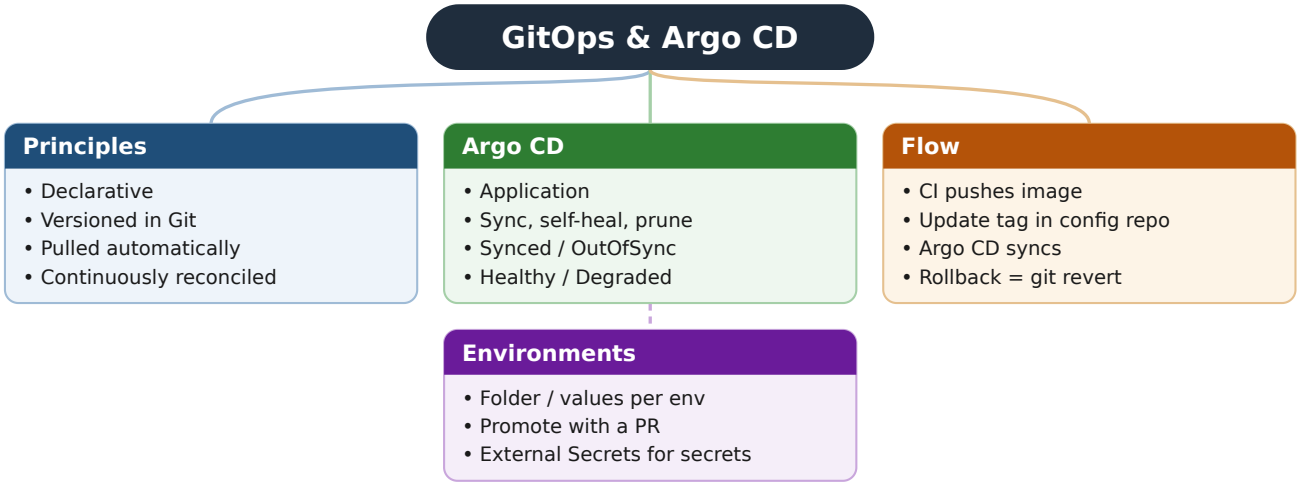
15. GitOps & Argo CD

DevOps, Cloud & Platform Engineering · Complete Notes · Topic 15 of 18 · Thirupathi Keloth · v2.0

Contents

15.0 Why GitOps
15.1 Principles
15.2 Repository design
15.3 Argo CD architecture
15.4 The Application resource
15.5 Sync waves, phases and hooks
15.6 Projects, RBAC and multi-tenancy
15.7 Scaling: App of Apps and ApplicationSets
15.8 Secrets in GitOps
15.9 Image updates and environment promotion
15.10 Progressive delivery with Argo Rollouts
15.11 Notifications
15.12 Flux CD
15.13 Operating GitOps in production
15.14 Troubleshooting
15.15 Hands-on labs (kind: free)
15.16 Interview questions
15.17 Self-check



15.0 Why GitOps

With classic CI/CD, the pipeline **pushes** changes into the cluster (`kubectl apply` or `helm upgrade` from CI). That means CI holds powerful cluster credentials, nobody knows whether the cluster still matches what was deployed (someone ran `kubectl edit` at 2 a.m.), and rebuilding a cluster means re-running dozens of pipelines.

GitOps makes Git the single source of truth for the desired state of the system, and an agent **inside** the cluster continuously pulls that state and makes reality match it. Every change is a commit: reviewed, audited, and revertible.

15.1 Principles

The OpenGitOps project (CNCF) defines four principles:

Principle	Meaning
1. **Declarative**	The whole desired state is expressed declaratively (Kubernetes YAML, Helm values, Kustomize)
2. **Versioned and immutable**	Stored in Git (or OCI), with full history; every version can be recovered
3. **Pulled automatically**	Software agents pull the desired state from the source; no external system pushes
4. **Continuously reconciled**	Agents constantly compare desired vs actual state and correct drift

Push vs pull deployment

```
PUSH (classic CI/CD)                              PULL (GitOps)
CI ──kubectl/helm (cluster-admin creds)──► K8s    CI ──commit new tag──► config repo (Git)
                                                                          ▲  pulls / watches
cluster drift goes unnoticed                      Argo CD / Flux in the cluster ──applies──► K8s
                                                  drift detected and corrected automatically
```

Benefit	Why
Audit trail	Who changed what, when, and who approved it: the Git log and PRs
Rollback	`git revert` and the cluster follows
Security	CI never needs cluster credentials; the cluster pulls outbound only
Drift detection	Manual changes are visible (OutOfSync) and can be auto-reverted (self-heal)
Disaster recovery	A new cluster + the GitOps agent pointed at the repo = everything restored
Consistency	Same process for every team, environment and cluster

Limits to know: GitOps manages *desired state*, not one-off actions (database migrations need hooks or Jobs); secrets need special handling (15.8); fast "click to rollback" becomes "revert a commit", so the review flow must be quick for emergencies.

15.2 Repository design

Separate application code from deployment configuration

Repository	Contains	Changed by
App repo (`shop-api`)	Source code, Dockerfile, tests, CI workflow, maybe a Helm chart	Developers
Config repo (`shop-gitops`)	Kubernetes manifests / Helm values / Kustomize overlays per environment, Argo CD Applications	CI (image tag bumps), platform and app teams via PRs

Why separate: CI writing tag bumps into the app repo would trigger more CI runs; config changes (replicas, env vars) shouldn't require an app build; access control differs (prod config needs stricter review).

Environments: folders, not branches

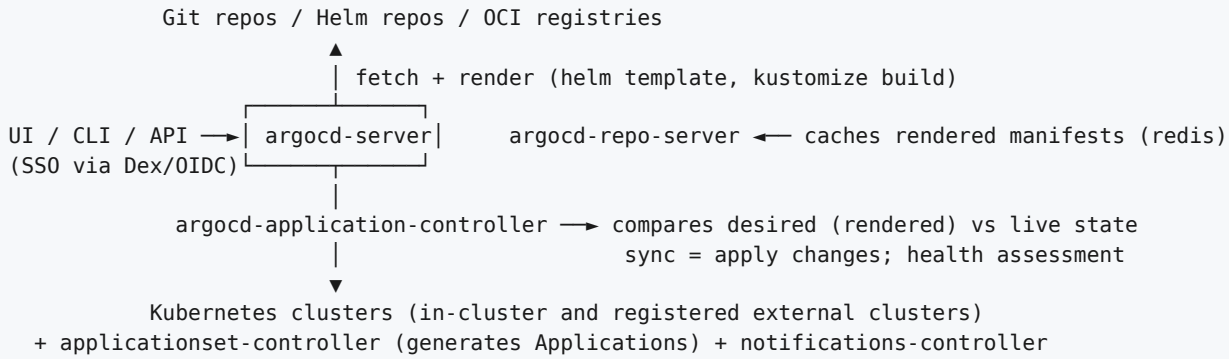
```
shop-gitops/
├── apps/
│   └── shop-api/
│       ├── base/                    # Deployment, Service, HPA... (Kustomize base)
│       └── overlays/
│           ├── dev/                 # image tag, replicas, config for dev
│           ├── staging/
│           └── prod/
├── platform/                        # cluster add-ons: ingress, cert-manager, ESO, monitoring
│   ├── cert-manager/
│   └── external-secrets/
└── clusters/
    ├── dev/                         # what runs in the dev cluster (Applications / AppSets)
    └── prod/
```

Avoid one branch per environment (`dev`, `staging`, `prod` branches): merges between branches drift, cherry-picks get lost, and you can't see all environments' differences in one place. Use **one main branch with a folder per environment**; promotion = a PR changing the next environment's folder.

Monorepo (one config repo for everything) is simpler to start; split per team or per domain when access control or scale requires it.

15.3 Argo CD architecture

Argo CD (CNCF graduated) is the most widely used GitOps tool for Kubernetes, with a web UI, CLI and API.



Component	Role
`argocd-server`	API, web UI, authentication (local users, SSO through Dex or direct OIDC)
`argocd-repo-server`	Clones repos and renders manifests (plain YAML, Helm, Kustomize, Jsonnet, config management plugins)
`argocd-application-controller`	Watches Applications, diffs desired vs live, syncs, computes health
`argocd-applicationset-controller`	Templates many Applications from generators
`argocd-notifications-controller`	Sends events to Slack, email, webhooks
Redis	Cache

Install and first login

```
kubectl create namespace argocd
kubectl apply -n argocd --server-side --force-conflicts \
  -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl -n argocd rollout status deploy/argocd-server
kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath='{.data.password}' | base64 -d; echo          # initial admin password
kubectl -n argocd port-forward svc/argocd-server 8443:443   # UI at https://localhost:8443
argocd login localhost:8443 --username admin --insecure     # lab only: self-signed cert
argocd account update-password
kubectl -n argocd delete secret argocd-initial-admin-secret # after changing the password
```

Production installs use the **Helm chart** (`argo/argo-cd`) with HA mode, SSO, ingress with TLS, and the Argo CD configuration itself managed by Argo CD (self-management).

15.4 The Application resource

An **Application** connects a **source** (repo + path + revision) to a **destination** (cluster + namespace), with a sync policy.

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: shop-api-prod
  namespace: argocd
  finalizers:
    - resources-finalizer.argocd.argoproj.io   # deleting the App deletes its resources
spec:
  project: shop
  source:
    repoURL: https://github.com/acme/shop-gitops.git
    targetRevision: main
    path: apps/shop-api/overlays/prod          # Kustomize detected automatically
  destination:
    server: https://kubernetes.default.svc     # the cluster Argo CD runs in
    namespace: shop
  syncPolicy:
    automated:
      prune: true                              # delete resources removed from Git
      selfHeal: true                           # revert manual changes in the cluster
    syncOptions:
      - CreateNamespace=true
      - ServerSideApply=true
      - PruneLast=true                         # prune after everything else is healthy
    retry:
      limit: 5
      backoff:
        duration: 10s
        factor: 2
        maxDuration: 3m
  ignoreDifferences:                           # fields owned by other controllers
    - group: apps
      kind: Deployment
      jsonPointers:
        - /spec/replicas                       # HPA manages replicas
  revisionHistoryLimit: 10
```

```
# A Helm chart from a chart repository, values from the config repo (multiple sources)
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: cert-manager
  namespace: argocd
spec:
  project: platform
  sources:
    - repoURL: https://charts.jetstack.io
      chart: cert-manager
      targetRevision: v1.16.1                 # pinned chart version
      helm:
        valueFiles:
          - $values/platform/cert-manager/values-prod.yaml
    - repoURL: https://github.com/acme/shop-gitops.git
      targetRevision: main
      ref: values                             # referenced above as $values
  destination:
    server: https://kubernetes.default.svc
    namespace: cert-manager
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

Status: sync and health

Sync status	Meaning
`Synced`	Live state matches Git at the target revision
`OutOfSync`	Differences exist (new commit, or drift)
Unknown	Couldn't compare (repo error, rendering error)

Health status	Meaning
`Healthy`	Resources are working (Deployment rolled out, pods ready)
`Progressing`	Not yet healthy but making progress (rollout in progress)
`Degraded`	Failed (rollout failed, CrashLoopBackOff, failed Job)
`Suspended`	Paused (suspended CronJob, paused Rollout)
`Missing`	Resource in Git but not in the cluster

Argo CD has built-in health checks for standard kinds and many CRDs; custom ones are written in Lua in `argocd-cm`.

Sync policy choices

Setting	Trade-off
Manual sync	A human clicks "Sync": control, but drift accumulates and it's easy to forget
`automated`	Commits deploy within minutes (default poll 3 minutes; Git webhooks make it near-instant)
`prune: true`	Removing YAML removes resources (otherwise orphans stay)

Setting	Trade-off
`selfHeal: true`	Manual `kubectl` changes are reverted; Git is enforced. Emergency change = commit (or temporarily disable auto-sync)

```
argocd app list
argocd app get shop-api-prod                  # status, health, resources, sync result
argocd app diff shop-api-prod                 # what would change
argocd app sync shop-api-prod [--prune] [--dry-run]
argocd app history shop-api-prod
argocd app rollback shop-api-prod 12          # only with auto-sync disabled; prefer git revert
argocd app set shop-api-prod --sync-policy none   # pause automation (emergency)
argocd app delete shop-api-prod
```

Rollback in GitOps = `git revert <commit>` in the config repo. `argocd app rollback` is a temporary measure: with automated sync on, Argo CD would re-apply Git again, so it requires disabling auto-sync, and Git must be fixed afterwards.

15.5 Sync waves, phases and hooks

Order resources within a sync with the annotation `argocd.argoproj.io/sync-wave` (lower numbers first, can be negative; Argo CD waits for each wave to be healthy). **Hooks** run at specific phases.

Phase	Use
`PreSync`	Database migration, backup before deploying
`Sync`	Normal resources (default)
`PostSync`	Smoke tests, notifications after everything is healthy
`SyncFail`	Cleanup or alert when a sync fails

```
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migrate
  annotations:
    argocd.argoproj.io/hook: PreSync
    argocd.argoproj.io/hook-delete-policy: BeforeHookCreation,HookSucceeded
    argocd.argoproj.io/sync-wave: "-1"
spec:
  backoffLimit: 1
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: migrate
          image: 123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/api:1.4.2
          command: ["python", "-m", "app.migrate"]
```

Typical waves: `-2` namespaces and CRDs → `-1` config, secrets (ExternalSecrets) → `0` workloads → `1` ingress/routes. Migrations must be **backward-compatible** (expand → migrate → contract, CI/CD notes 11.4) because the old version keeps running during rollout.

15.6 Projects, RBAC and multi-tenancy

An **AppProject** limits what a team's Applications can do: which repos, which clusters/namespaces, which resource kinds.

```
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: shop
  namespace: argocd
spec:
  description: Shop team applications
  sourceRepos:
    - https://github.com/acme/shop-gitops.git
  destinations:
    - server: https://kubernetes.default.svc
      namespace: shop
    - server: https://kubernetes.default.svc
      namespace: shop-*
  clusterResourceWhitelist: []                 # no cluster-scoped resources
  namespaceResourceBlacklist:
    - group: ""
      kind: ResourceQuota                      # quotas are owned by the platform team
  roles:
    - name: developers
      description: Shop developers can sync their apps
      policies:
        - p, proj:shop:developers, applications, get, shop/*, allow
        - p, proj:shop:developers, applications, sync, shop/*, allow
      groups:
        - shop-developers                      # SSO group
  syncWindows:
    - kind: deny                               # no automatic prod syncs during peak sale hours
      schedule: "0 18 * * *"
      duration: 4h
      applications: ["*-prod"]
```

Global RBAC lives in the `argocd-rbac-cm` ConfigMap (Casbin policy lines: `p, <subject>, <resource>, <action>, <object>, allow|deny` and `g, <group>, <role>`), with `policy.default: role:readonly` or stricter. Use **SSO groups**, not local users; disable the `admin` user after setup.

Never let application teams use the built-in `default` project in production: it allows any repo and any destination.

15.7 Scaling: App of Apps and ApplicationSets

App of Apps (bootstrap pattern)

One root Application points at a folder full of Application manifests; Argo CD creates them all. Bootstrapping a cluster becomes: install Argo CD, apply one root Application.

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: root-prod
  namespace: argocd
spec:
  project: default                              # platform-owned bootstrap only
  source:
    repoURL: https://github.com/acme/shop-gitops.git
    targetRevision: main
    path: clusters/prod                         # contains Application/ApplicationSet YAMLs
  destination:
    server: https://kubernetes.default.svc
    namespace: argocd
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

ApplicationSet

Generates Applications from **generators**, replacing hand-written copies:

Generator	Creates one Application per…
List	Item in a static list
Cluster	Registered cluster (optionally filtered by labels): deploy add-ons to every cluster
Git directory / file	Folder (or config file) in a repo: a new folder = a new app
Matrix / Merge	Combination of two generators (every app × every cluster)
Pull request	Open PR: ephemeral preview environments
SCM provider	Repository in a GitHub/GitLab organization

```
# Every folder under apps/*/overlays/dev becomes an Application in the dev cluster
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: dev-apps
  namespace: argocd
spec:
  goTemplate: true
  goTemplateOptions: ["missingkey=error"]
  generators:
    - git:
        repoURL: https://github.com/acme/shop-gitops.git
        revision: main
        directories:
          - path: apps/*/overlays/dev
  template:
    metadata:
      name: "{{ index .path.segments 1 }}-dev"   # apps/<name>/overlays/dev -> <name>-dev
    spec:
      project: shop
      source:
        repoURL: https://github.com/acme/shop-gitops.git
        targetRevision: main
        path: "{{ .path.path }}"
      destination:
        name: dev-cluster
        namespace: "{{ index .path.segments 1 }}"
      syncPolicy:
        automated:
          prune: true
          selfHeal: true
        syncOptions:
          - CreateNamespace=true
```

```yaml
# Install the monitoring stack on every cluster labeled env=prod
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: monitoring
  namespace: argocd
spec:
  goTemplate: true
  generators:
    - clusters:
        selector:
          matchLabels:
            env: prod
  template:
    metadata:
      name: "monitoring-{{ .name }}"
    spec:
      project: platform
      source:
        repoURL: https://github.com/acme/shop-gitops.git
        targetRevision: main
        path: platform/monitoring
      destination:
        server: "{{ .server }}"
        namespace: monitoring
      syncPolicy:
        automated:
          prune: true
          selfHeal: true
        syncOptions:
          - CreateNamespace=true
          - ServerSideApply=true
```

Multi-cluster

`argocd cluster add <kube-context> --label env=prod` registers a cluster (Argo CD creates a ServiceAccount there and stores credentials as a Secret in `argocd`). On EKS, prefer IAM-based cluster secrets (IRSA/Pod Identity for Argo CD) over long-lived tokens. Topologies: **hub-and-spoke** (one central Argo CD manages many clusters: one pane of glass, but a big blast radius) or **one Argo CD per cluster** (isolation, more instances to run). Many organizations use one per environment tier.

15.8 Secrets in GitOps

Plain Kubernetes Secrets must never be committed. Options (details in Kubernetes notes 12.8):

Approach	What's in Git
External Secrets Operator (recommended on AWS)	`ExternalSecret` references to Secrets Manager / Parameter Store paths
Sealed Secrets	`SealedSecret` encrypted with the cluster's public key
SOPS + KMS/age	Encrypted values; decrypted by a plugin (Argo CD CMP or Flux native SOPS support)
Secrets Store CSI driver	Only `SecretProviderClass` references

Avoid plugins that inject secret values into rendered manifests inside Argo CD (they end up in Argo CD's cache and UI diffs); prefer operators that create Secrets in the cluster.

15.9 Image updates and environment promotion

The end-to-end flow

```
developer PR ─► merge to main (app repo) ─► CI: test, build, scan, push image sha-3f9a1c2, sign
                                                  │
                                                  ▼ CI commits (or opens a PR) in the config repo
              apps/shop-api/overlays/dev/kustomization.yaml: newTag: sha-3f9a1c2
                                                  │
                                                  ▼ Argo CD syncs dev automatically
              tests pass in dev ─► PR: copy the same tag to staging ─► merge ─► staging syncs
              staging verified  ─► PR: copy the same tag to prod (CODEOWNERS approval) ─► prod
```

The **same image digest** moves through environments; only configuration differs. Each promotion is a reviewed, revertible commit.

```
# What CI runs in the config repo (job with a token scoped to that repo only)
cd apps/shop-api/overlays/dev
kustomize edit set image shop/api="${REGISTRY}/shop/api:${TAG}"
git commit -am "shop-api: dev -> ${TAG}"
git push                      # or: push a branch and open a PR for staging/prod
```

Automation options

Tool	How
CI commit / PR (above)	Simple, explicit, works with any tool
Argo CD Image Updater	Watches the registry for new tags matching a rule (semver, latest build), writes the new tag back to Git (or parameter overrides)
Kargo	Promotion orchestration on top of Argo CD: "freight" (image + config versions) moves through stages with verification
Flux image automation	`ImageRepository` + `ImagePolicy` + `ImageUpdateAutomation` commit tag bumps to Git
Renovate	Opens PRs for new image/chart versions in the config repo

15.10 Progressive delivery with Argo Rollouts

A Kubernetes Deployment only does rolling updates. **Argo Rollouts** replaces the Deployment with a `Rollout` resource that supports **canary** and **blue-green** with automated analysis of metrics, and automatic rollback.

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: shop-api
  namespace: shop
spec:
  replicas: 5
  revisionHistoryLimit: 5
  selector:
    matchLabels:
      app.kubernetes.io/name: shop-api
  template:
    metadata:
      labels:
        app.kubernetes.io/name: shop-api
    spec:
      containers:
        - name: app
          image: 123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/api:1.4.2
          ports:
            - containerPort: 8080
  strategy:
    canary:
      canaryService: shop-api-canary           # Services managed by Rollouts
      stableService: shop-api-stable
      trafficRouting:
        nginx:
          stableIngress: shop-api               # or ALB, Istio, Gateway API plugin...
      analysis:                                 # runs in the background during the canary
        templates:
          - templateName: success-rate
        startingStep: 1
        args:
          - name: service
            value: shop-api-canary
      steps:
        - setWeight: 10
        - pause:
            duration: 5m
        - setWeight: 30
        - pause:
            duration: 5m
        - setWeight: 60
        - pause:
            duration: 10m
        # then 100%: promotion complete
---
apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata:
  name: success-rate
  namespace: shop
spec:
  args:
    - name: service
  metrics:
    - name: success-rate
      interval: 1m
      count: 5
      failureLimit: 1                           # more than one failed measurement = abort
      successCondition: result[0] >= 0.99
      provider:
        prometheus:
          address: http://prometheus-operated.monitoring:9090
          query: |
            sum(rate(http_requests_total{service="{{args.service}}",status!~"5.."}[2m]))
```

```
            /
            sum(rate(http_requests_total{service="{{args.service}}"}[2m]))
```

If the success rate drops below 99%, the Rollout **aborts** and shifts all traffic back to stable automatically. Blue-green: `strategy.blueGreen` with `activeService`, `previewService`, optional pre/post-promotion analysis and `autoPromotionEnabled: false` for a manual switch.

```
kubectl argo rollouts get rollout shop-api -n shop --watch
kubectl argo rollouts promote shop-api -n shop       # skip the current pause
kubectl argo rollouts abort shop-api -n shop         # back to stable
```

Alternative: **Flagger** (works with Flux or Argo CD, with service meshes and ingress controllers).

15.11 Notifications

The notifications controller sends events (sync succeeded/failed, health degraded, new deployment) to Slack, Teams, email, GitHub commit statuses, webhooks.

```
# Subscribe one Application to Slack messages (triggers and templates come from the catalog)
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: shop-api-prod
  namespace: argocd
  annotations:
    notifications.argoproj.io/subscribe.on-sync-failed.slack: shop-deploys
    notifications.argoproj.io/subscribe.on-health-degraded.slack: shop-deploys
    notifications.argoproj.io/subscribe.on-deployed.slack: shop-deploys
spec:
  project: shop
  source:
    repoURL: https://github.com/acme/shop-gitops.git
    targetRevision: main
    path: apps/shop-api/overlays/prod
  destination:
    server: https://kubernetes.default.svc
    namespace: shop
```

The Slack token lives in the `argocd-notifications-secret` Secret (populated from your secret store), never in Git. Also send deploy events to Grafana as annotations so dashboards show "what changed".

15.12 Flux CD

Flux (CNCF graduated) is the other major GitOps tool: a set of Kubernetes controllers (the GitOps Toolkit), no built-in UI, very Kubernetes-native and composable.

```yaml
apiVersion: source.toolkit.fluxcd.io/v1
kind: GitRepository
metadata:
  name: shop-gitops
  namespace: flux-system
spec:
  interval: 1m
  url: https://github.com/acme/shop-gitops.git
  ref:
    branch: main
---
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
  name: shop-api-prod
  namespace: flux-system
spec:
  interval: 10m
  sourceRef:
    kind: GitRepository
    name: shop-gitops
  path: ./apps/shop-api/overlays/prod
  prune: true
  wait: true                                    # wait for resources to be ready
  timeout: 5m
---
apiVersion: source.toolkit.fluxcd.io/v1
kind: HelmRepository
metadata:
  name: jetstack
  namespace: flux-system
spec:
  interval: 1h
  url: https://charts.jetstack.io
---
apiVersion: helm.toolkit.fluxcd.io/v2
kind: HelmRelease
metadata:
  name: cert-manager
  namespace: cert-manager
spec:
  interval: 30m
  chart:
    spec:
      chart: cert-manager
      version: v1.16.1
      sourceRef:
        kind: HelmRepository
        name: jetstack
        namespace: flux-system
  values:
    crds:
      enabled: true
```

```
flux bootstrap github --owner=acme --repository=shop-gitops --path=clusters/prod
flux get kustomizations
flux reconcile kustomization shop-api-prod --with-source
flux logs --level=error
```

	Argo CD	**Flux**
UI	Rich web UI (big adoption driver)	None built in (Weave GitOps / Headlamp / Capacitor plugins)

	Argo CD	Flux
Model	`Application` / `ApplicationSet`	`GitRepository` / `OCIRepository` + `Kustomization` / `HelmRelease`
Helm	Renders with `helm template` (no Helm release records)	Real Helm releases (helm list works, Helm hooks native)
Multi-tenancy	AppProjects + RBAC + SSO in Argo CD	Kubernetes RBAC + service account impersonation per Kustomization
Multi-cluster	Hub-and-spoke from one instance is easy	Usually Flux per cluster
Secrets	Via operators/plugins	Native SOPS decryption
Image automation	Image Updater (separate)	Built-in controllers
Typical choice	Teams wanting visibility and self-service UI	Platform teams wanting composable, API-only GitOps

Both are solid; pick one per organization. The concepts (repo layout, promotion, secrets, drift) are the same.

15.13 Operating GitOps in production

Area	Practice
Availability	Argo CD HA manifests/Helm values (multiple replicas of server/repo-server, Redis HA); controller sharding for many clusters
Performance	Git webhooks instead of polling; resource exclusions for noisy CRDs; increase repo-server parallelism for large monorepos; `ApplyOutOfSyncOnly=true` for huge apps
Upgrades	Read release notes (Argo CD 3.x changed defaults: annotation-based resource tracking, stricter RBAC for logs and sub-resources); upgrade one minor at a time; manage Argo CD with Argo CD
Security	SSO only, `admin` disabled, AppProjects per team, restricted cluster-scoped resources, repo credentials as scoped deploy keys/GitHub App, network access to the API restricted, audit Git and Argo CD events
Disaster recovery	Everything is in Git: new cluster → install Argo CD → apply root app. Back up Argo CD's own config (`argocd admin export`) for credentials/cluster registrations not in Git
Break-glass	Documented emergency procedure: disable auto-sync for one app, apply the fix, then commit the same fix to Git and re-enable
Policy	Validate manifests in the config repo's CI (kubeconform, Kyverno CLI, conftest) so bad YAML never reaches Argo CD
Drift reporting	Alert on apps `OutOfSync` or `Degraded` for more than N minutes (Argo CD exposes Prometheus metrics: `argocd_app_info{sync_status,health_status}`)

```
# Applications degraded or out of sync (alert if this stays > 0 for 15 minutes)
count by (name, project) (argocd_app_info{health_status!="Healthy"})
count by (name, project) (argocd_app_info{sync_status="OutOfSync"})
```

15.14 Troubleshooting

Symptom	Likely cause	Fix
`ComparisonError` / `Unknown`	Repo unreachable or credentials wrong; Helm/Kustomize render failure; bad path	`argocd app get` message; test `kustomize build` / `helm template` locally; check repo-server logs
Always `OutOfSync` right after syncing	Fields mutated by controllers/webhooks (replicas by HPA, defaults added by admission, CRD defaulting)	`argocd app diff`; `ignoreDifferences` (+ `RespectIgnoreDifferences=true`), ServerSideApply, normalize manifests
Sync succeeded but app `Degraded`	Pods failing (image, config, probes)	Kubernetes troubleshooting (12.18) on the resources Argo CD shows
`Progressing` forever	Rollout waiting on readiness, a LoadBalancer Service waiting for an address, missing health check for a CRD	Inspect the resource; add a custom health check
Sync stuck "waiting for completion of hook"	PreSync Job failing or hanging	Job logs; `hook-delete-policy`; terminate the operation (`argocd app terminate-op`)
Resources not deleted	`prune` disabled, or resource has `Prune=false` annotation	Enable prune; sync with `--prune`
Manual changes keep reverting	`selfHeal: true` is working as designed	Change Git instead
Application won't delete	Finalizer waiting for resource deletion	Check the resources/their finalizers; remove the App finalizer only knowingly
`permission denied` / "not permitted in project"	AppProject `sourceRepos` / `destinations` /whitelists or RBAC	Update the project or RBAC policy
Changes take minutes to appear	Polling interval (3 min default)	Configure Git webhooks to `argocd-server`
Secret values missing	ExternalSecret not syncing (store auth, wrong key)	`kubectl describe externalsecret`; ESO logs

15.15 Hands-on labs (kind: free)

#	Lab
1	Install Argo CD in kind; log in via port-forward and CLI; change the admin password
2	Create your own config repo with a Kustomize base and dev/prod overlays for a small app; create an Application for dev (manual sync), sync it
3	Turn on automated sync with prune and self-heal; `kubectl scale` the Deployment manually and watch Argo CD revert it; delete a YAML file and watch prune
4	Promote: change the image tag in dev via commit, then promote to prod via a PR; roll back with `git revert`
5	Add a PreSync migration Job and sync waves; observe the order in the UI
6	Create an AppProject restricting repos and namespaces; try to deploy outside it and read the error

#	Lab
7	Replace individual Applications with an App of Apps, then with a Git-directory ApplicationSet; add a new app by adding a folder
8	Install a Helm chart (e.g. cert-manager) as a multi-source Application with values from your repo
9	Install Argo Rollouts; convert the Deployment to a canary Rollout; promote and abort manually
10	Add External Secrets with a fake/local provider or Sealed Secrets so no plain Secret is in Git
11	Repeat lab 2–3 with Flux (`flux bootstrap` against your GitHub repo) and compare the experience
12	Disaster drill: delete the kind cluster, recreate it, install Argo CD, apply the root app; time the full recovery

15.16 Interview questions

1. What is GitOps? State the four OpenGitOps principles.
2. Push-based CI/CD vs pull-based GitOps: security and operational differences.
3. Why separate application and configuration repositories?
4. Why are environment branches an anti-pattern? How do you model environments?
5. Explain Argo CD's architecture and what each component does.
6. What does an Application define? Sync status vs health status.
7. What do `prune` and `selfHeal` do, and what are the risks of each?
8. How do you roll back in a GitOps world? Why is `argocd app rollback` not enough?
9. An app is always OutOfSync after sync: causes and fixes.
10. How do you run database migrations with Argo CD (hooks, waves, backward compatibility)?
11. What is an AppProject? How do you make Argo CD safe for many teams?
12. App of Apps vs ApplicationSet; name ApplicationSet generators and use cases.
13. How do you manage add-ons across 20 clusters?
14. How do you handle secrets with GitOps?
15. Describe the full flow from a developer's merge to production with GitOps, including promotion.
16. How does Argo Rollouts canary with analysis work? What happens when metrics are bad?
17. Argo CD vs Flux: differences and when you'd choose each.
18. How do you handle an emergency fix when self-heal reverts manual changes?
19. How do you monitor Argo CD and alert on failed deployments?
20. How would you rebuild a production cluster from scratch with GitOps, and how long should it take?

15.17 Self-check

- [ ] I can explain GitOps principles and the push vs pull model with its security benefits.
- [ ] I can design a config repository with environment folders and a promotion process.
- [ ] I can install Argo CD and write Applications for Kustomize, Helm and multi-source setups.
- [ ] I can choose sync options (automated, prune, self-heal, waves, hooks, ignoreDifferences) deliberately.
- [ ] I can secure multi-tenancy with AppProjects, RBAC and SSO.
- [ ] I can bootstrap clusters with App of Apps and scale with ApplicationSets.
- [ ] I can keep secrets out of Git with ESO or Sealed Secrets.
- [ ] I can implement canary releases with Argo Rollouts and metric analysis.
- [ ] I can compare Argo CD and Flux and troubleshoot common sync and health problems.

REMEMBER

- **Git is the source of truth.** Nobody runs `kubectl apply` on production by hand.